

Harshit Kandpal

kandpalhar@gmail.com • HarK-github • [harshit-k-a746a1310](#) • +91 9321763273 • Durg,
India (IST/UTC+5:30)

Cover Letter: Hyperledger Fabric-X + Fablo Integration

To: OpenSSF Mentorship Program Committee

Mentors: Umegbewe 'Great' Nwebedu, Osama Rabea, and Jakub Dzikowski

Date: May 2026

It is evident that Hyperledger Fabric-X presents a significant evolution in tokenized asset management on the blockchain, extending the capabilities of traditional Hyperledger Fabric with its modular architecture and enhanced privacy features. However, effectively deploying and managing these Fabric-X networks still poses a non-trivial challenge, often requiring extensive manual configuration and specialized tooling. The current Fablo framework excels at simplifying classic Fabric deployments through intuitive JSON configurations and automated Docker Compose generation, yet extending this seamless experience to Fabric-X requires careful consideration of its distinct multi-component architecture. The challenge lies in harmonizing Fablo's declarative simplicity with Fabric-X's inherent complexities, such as its three-stage crypto pipeline, multiple topology modes (e.g., 'xdev' versus 'decomposed'), and granular microservice configurations.

Integrating Fabric-X into Fablo is not merely about adding a new set of Docker containers; it necessitates a deep understanding of how to abstract Fabric-X's unique identity management, FSC node identities, and Token SDK namespace generation into a flexible, configurable model. The current 'fablo-repo' architecture, while robust for classic Fabric, needs strategic extensions to accommodate Fabric-X's specific requirements without compromising backward compatibility. This involves developing a dedicated Fabric-X generator within Fablo that can dynamically synthesize crypto material using 'cryptogen' and 'fxconfig', manage orderer and committer microservice configurations, and support varied deployment topologies. The objective is to transition Fabric-X setup from a manual, sample-dependent process to a fully automated, configurable one, addressing issues like dynamic crypto generation, scalable orderer identity management, and support for complex, decomposed network topologies.

My technical background and hands-on experience are directly aligned with the demands of this integration project. Over the past two years, I have gained substantial experience in distributed systems, Docker-based tooling, TypeScript/Node.js development, and robust CI/CD workflows through numerous open-source contributions. My work on blockchain projects, such as the decentralized article-publishing platform AuthrNet, has provided me with a strong familiarity with token-style and permissioned workflows that mirror Fabric-X's namespace-based model. Crucially, I have already invested significant time in studying both Fablo's architecture and the Fabric-X token sample end-to-end, leading to the development of a working MVP. This proof-of-concept, hosted at '<https://github.com/HarK-github/FabloFabricxIntegration>', demonstrates a 'global.platform: "fabricx"' generator branching and functional 'fablo init -fabricx', 'fablo status', and 'fablo down' commands for a bundled Fabric-X network. This proactive engagement underscores my readiness to tackle the intricacies of this project.

I am particularly eager to collaborate with and learn from experienced mentors such as Umegbewe 'Great' Nwebedu, Osama Rabea, and Jakub Dzikowski. Their profound expertise in Hyperledger technologies and open-source development will be invaluable as I navigate the complexities of this integration, ensuring that the solutions developed are not only functional but also adhere to best practices and community standards. My commitment extends beyond merely implementing features; I am dedicated to delivering a robust, well-tested, and fully documented Fabric-X integration into Fablo that truly simplifies its adoption for developers worldwide. I am confident that my blend of foundational knowledge, direct project experience, and passion

for open-source innovation makes me an ideal candidate to bring this critical integration to fruition.

How I Found Out About the Mentorship Program

I discovered the OpenSSF LFX Mentorship Program directly through the LFDT website, where I actively search for structured open-source opportunities. The program's focus on impactful projects within supply chain security, like the Hyperledger Fabric-X and Fablo integration, immediately captured my attention. I find these programs crucial for gaining hands-on experience in real-world infrastructure rather than isolated academic exercises.

Why I Am Interested in This Program

My interest stems from a long-standing engagement with open-source and decentralized technologies, seeking to apply my skills to impactful hands-on development. The Fablo project's role in lowering the barrier to entry for Hyperledger Fabric-X deeply resonates with me, highlighting its community importance. This ambitious integration offers a significant learning curve, promising substantial growth in my understanding of complex systems and the satisfaction of contributing to a tool that benefits many.

Experience and Knowledge/Skills Applicable to This Program

My two years of open-source contributions, marked by over 800 GitHub contributions, have provided extensive experience in distributed systems, Docker-based tooling, TypeScript/Node.js, and CI/CD workflows, all directly relevant to this project. I have leveraged Docker and GitHub Actions for containerized applications and multi-service orchestration, crucial for managing Fabric-X networks via Docker Compose and Fablo. Specifically, my prior study of Fablo's architecture and the Fabric-X token sample, including its xdev topology, core.yaml, routing tables, and fxconfig lifecycle, culminated in a functional MVP implementing 'global.platform: "fabricx"' generator branching and network lifecycle commands. This focused work positions me exceptionally well to drive the Fabric-X integration into Fablo.

What I Hope to Get Out of This Mentorship Experience

Through this mentorship, I aim to significantly enhance my proficiency with complex codebase architectures and deepen my understanding of both Fablo and Fabric-X. I particularly look forward to gaining advanced confidence in Docker-based environments and microservice-oriented systems. Learning directly from the mentors' real-world experience and guidance is paramount for my development as an effective open-source contributor, enabling me to make lasting, meaningful contributions to the community and the project's future evolution.

Sincerely,

Harshit Kandpal

B.Tech Computer Science, IIT Bhilai (2024–2028)

PROPOSAL

OpenSSF LFX Mentorship 2026 – Summer Term

Hyperledger Fabric-X + Fablo Integration

Project: Hyperledger Fabric-X + Fablo Integration

Mentors: Umegbewe 'Great' Nwebedu, Osama Rabea, and Jakub Dzikowski

Duration: Mid-June to end of November 2026 (part-time, 12–15 hrs/week)

Harshit Kandpal

B.Tech Computer Science, IIT Bhilai (2024–2028)

HarK-github

Durg, India (IST/UTC+5:30)

kandpalhar@gmail.com

Index / Table of Contents

Contents

1	Executive Summary	5
2	The Problem: Why This Work Matters	5
2.1	The Fabric-X developer experience gap today	6
2.2	What Fablo does today and why it is the right foundation	6
2.3	Why the integration is non-trivial	7
3	Technical Background and Research	7
3.1	Fablo codebase analysis	7
3.2	Fabric-X architecture analysis	8
4	Proposed Solution: The Pluggable Engine Architecture	9
4.1	Integration path evaluation and decision	9
4.2	Schema-first engine detection	9
4.3	The FabloEngine interface	11
4.4	The config schema design	11
4.5	Generated artifacts and docker-compose structure	12
5	MVP Plan and Timeline	13
5.1	Phase plan and milestone table	13
5.2	Gantt placeholder	14

6	Full Technical Approach: Beyond the MVP	14
6.1	Crypto material generation for Fabric-X	14
6.2	Multi-service topology support	15
6.3	CCaaS and endorser integration	16
6.4	Full e2e test suite	17
6.5	Lifecycle management: reset and channel tooling	19
7	Known Limitations and How They Will Be Resolved	20
8	Pre-Application Research	21
9	Why I Am the Right Candidate	21
10	Risk Register	22
11	Success Metrics	22

1 Executive Summary

Hyperledger Fabric-X is not a minor release upgrade of Hyperledger Fabric. It is a different network architecture with a different operational center of gravity. In classic Fabric, Fablo can describe a network in terms of organizations, peers, orderers, certificate authorities, channels, and chaincodes, then generate a working Docker-based environment from a single `fablo-config.json`. Fabric-X changes those assumptions. The Fabric-X repository and sample deployment material show a decomposed system with separate orderer, sidecar, coordinator, verifier, validator-committer, query service, PostgreSQL, and Fabric Smart Client nodes. The bundled `xdev` sample in `fabric-x-repo/samples/tokens/compose-xdev.yml` is already enough to prove that running Fabric-X locally requires more than renaming classic Fabric containers. Its startup path depends on service-specific environment variables, routing tables, FSC node configuration, and crypto material that is currently assembled from sample scripts rather than from a tool like Fablo.

Fablo is the right foundation for solving this because it already owns the part of the developer experience that matters most: taking a compact declarative configuration and turning it into reproducible local infrastructure. The current codebase uses a Docker-based generation approach where the `setup-network` command runs inside a container to generate network artifacts using EJS templates. However, this monolithic architecture makes it difficult to support fundamentally different network types like Fabric-X without extensive modifications.

The solution is to introduce a pluggable engine architecture that separates network-type-specific logic into interchangeable engines. This MVP will implement the engine framework and a Fabric-X engine that reads a single `fablo-config-fabricx.json`, validates it against a versioned JSON schema, generates all Docker Compose and configuration artifacts, and bootstraps a working local Fabric-X network. The engine architecture will maintain backward compatibility with classic Fabric while enabling clean extension for new network types.

Core Deliverable

A pluggable engine architecture for Fablo with a production-quality Fabric-X engine that reads a single `fablo-config-fabricx.json`, validates it against a versioned JSON schema, generates all Docker Compose and configuration artifacts, and bootstraps a working local Fabric-X network with a single command — with full lifecycle management (`fablo up`, `fablo down`, `fablo status`, `fablo reset`), automated e2e tests mirroring Fablo's existing test structure, and contributor documentation.

The mentorship work will implement this engine architecture from scratch, starting with the current Docker-based Fablo codebase. The main themes are: introduce schema-first engine detection, isolate Fabric-X behavior in dedicated engine code, implement a clean local developer workflow for the bundled `xdev` topology, then extend toward richer topology, stronger crypto generation, and broader lifecycle coverage.

Stretch Goals (begin Week 13 only after core is merged)

Multi-org Fabric-X topologies; separate-container committer/endorser/orderer support; dynamic crypto generation pipeline.

support; CI integration example; Fablo UI support for Fabric-X configs.

The current code path from user input to running containers works as follows: A user calls `fablo up` `fablo-config.json`, which invokes the `fablo.sh` script. This script checks if network files exist in `fablo-target/`. If not, it runs the `setup-network` command inside a Docker container, mounting the config file and target directory. The `src/setup-docker/index.ts` command parses the config, extends it with `extendConfig()`, and generates all artifacts using EJS templates. Once generated, `fablo.sh` calls the generated `fablo-target/fabric-docker.sh` up script to start the network.

This Docker-based generation approach works well for classic Fabric but creates challenges for supporting different network types like Fabric-X. The monolithic `setup-docker/index.ts` contains logic specific to classic Fabric topology, making it difficult to extend for fundamentally different architectures without extensive conditional branching.

3.2 Fabric-X architecture analysis

Component	Role and config surface
Arma Orderer	The README describes an orderer stack made of routers, batchers, consenters, and assemblers. In the Ansible inventory, routers expose ports such as 7050, 7150, 7250, and 7350; batchers, consenters, and assemblers use adjacent ports.
Endorser	In the current MVP schema, endorser is an FSC node role. It receives a generated <code>core.yaml</code> with node identity, P2P endpoint, routing references, Fabric-X driver settings, and token namespace wiring.
Validator-Committer	The Ansible inventory models the validator as its own committer component with <code>committer_component_type: validator</code> and an RPC port such as 5100. It represents the validation and commitment half of the decomposed commit path.
Sidecar	The sidecar is the Fabric-facing entry point for FSC nodes. The xdev sample configures it with orderer identity variables and a public port 4001. The current engine points node <code>peers</code> entries at <code>localhost:{sidecar}</code> .
Query Service	The xdev sample exposes the query service at 7001 through <code>SC_QUERY_SERVICE_SERVER_ENDPOINT=:7001</code> . The current engine waits for this port on startup and places it under <code>queryService</code> in generated node config.
Coordinator	The Ansible inventory models a separate <code>coordinator</code> component with its own RPC port, such as 5300, and the xdev sample exposes a dedicated logging level variable for it.
Verifier	The verifier is also a distinct committer component in the inventory, typically with its own RPC port, such as 5200. The xdev sample exposes <code>SC_VERIFIER_LOGGING_LEVEL</code> .
PostgreSQL	The bundled xdev sample exposes PostgreSQL on 5433. The decomposed inventory models it separately as <code>committer-db</code> with user, password, database name, and a different port (5435) in that topology.

Table 1: Fabric-X decomposed components — confirmed from fabric-x-repo

My working MVP deliberately targets the bundled `xdev` topology because that is the smallest viable integration surface that still represents real Fabric-X behavior. The sample image in `compose-xdev.yaml` already bundles database, orderer, and committer behavior behind one container command, `run db orderer committer`. That is ideal for a first merge because it keeps the number of containers low while still requiring real Fabric-X configuration. The current Fablo engine maps onto this model directly: one generated compose file for the bundled infrastructure service and one `core.yaml` per logical node. This is enough to prove engine architecture, lifecycle delegation, template rendering, default port handling, and schema-first detection without taking on the full burden of a decomposed multi-service topology on day one.

At the same time, the local repository makes it clear that `xdev` is only a starting point. The Ansible inventory proves that Fabric-X's natural topology is decomposed, multi-process, and potentially multi-organization. The `tools/fxconfig` tree and its integration tests (`single_org_test.go` and `multi_org_test.go`) also show that configuration assembly and namespace lifecycle are already serious enough to justify dedicated tooling. That is why I should frame the MVP honestly: it is a local bootstrap path for the bundled developer topology, not the full end-state of Fabric-X operations.

[DIAGRAM PLACEHOLDER: Classic Fabric (peer + orderer + CA per org) vs. Fabric-X decomposed architecture (Arma orderer, endorser, validator-committer, sidecar, query service, coordinator, verifier)]

Figure 1: Classic Hyperledger Fabric vs. Fabric-X decomposed architecture

4 Proposed Solution: Implementing Pluggable Engine Architecture for Fabric-X

4.1 Integration path evaluation and decision

The pluggable engine is the correct architectural choice because it enables clean support for different network types while maintaining Fablo's established user experience. The current Docker-based generation approach works for classic Fabric but cannot easily accommodate Fabric-X's different topology and configuration requirements. By introducing a pluggable engine architecture, we can isolate network-type-specific logic while preserving the existing CLI interface and generation workflow.

This approach will implement the engine framework as part of the MVP, creating a foundation that supports both classic Fabric and Fabric-X. The engine architecture follows the Open/Closed Principle: the core command code remains closed for modification with respect to network type, but open for extension through new engine implementations.

4.2 Schema-first engine detection

The key design choice is to detect the engine from the configuration schema, not from a custom platform field. This approach eliminates ambiguous states where a config might be misinterpreted. The schema identifies both the config shape and the engine that must process it.

Engine detection logic (src/engines/engine-loader.ts)

```

1 export function getEngine(config: any): FabloEngine {
2   if (config === null || typeof config !== "object") {
3     throw new Error("Invalid config: expected a JSON object.");
4   }
5
6   if (typeof config.$schema === "string" &&
7       config.$schema.endsWith("fabricx-schema-v1.json")) {
8     return new FabricXEngine();
9   }
10  return new ClassicFabricEngine();
11 }

```

The suffix match is deliberate. Using `.endsWith()` instead of `.includes()` avoids false positives where an

Approach	Advantages	Disadvantages	Verdict
Separate repository	Total isolation from classic Fabric code; freedom to tailor commands, schemas, and templates only for Fabric-X; lower short-term coupling to historic Fablo assumptions.	Duplicates Fablo's CLI contract, target-directory handling, config parsing, docs surface, and test style; splits the contributor community; turns Fabric-X support into a parallel tool instead of an extension of the standard Fabric bootstrap workflow.	Rejected
Wrapper over Ansible	Fastest route to a demo if the only goal is to invoke existing sample tooling; preserves upstream sample scripts without deep integration work.	Leaves the developer with a two-layer mental model; errors surface through wrapper indirection; does not resolve schema mismatch; keeps ownership of generated artifacts outside Fablo; fails to create a real pluggable architecture.	Rejected
Pluggable engine in Fablo	Reuses mature commands, test patterns, target-dir logic, and documentation expectations; keeps classic behavior intact; isolates Fabric-X logic under <code>src/engines/fabricx/</code> ; allows a future third engine without changing core CLI commands.	Requires implementing the engine architecture from scratch; needs careful handling of the existing Docker-based generation approach.	Selected

Table 2: Integration approach evaluation

unrelated URL merely contains the Fabric-X schema name as a fragment or query component. Including `v1` in the schema filename creates a stable forward-compatibility story for future schema versions.

This schema-first detection will be implemented as part of the MVP, transitioning from the current monolithic Docker-based generation to a pluggable engine system. The new approach isolates engine selection to the schema itself, allowing the CLI commands to remain engine-agnostic while enabling clean code ownership for different network types.

4.3 The **FabloEngine** interface

The power of the engine approach comes from the fact that the interface is intentionally small and lifecycle-shaped rather than generation-shaped. This design matters because different network types have different startup requirements and lifecycle semantics.

Engine contract (src/engines/engine.ts)

```
1 export interface ValidationResult {
2   valid: boolean;
3   errors: string[];
4 }
5
6 export interface FabloEngine {
7   validate(rawConfig: unknown): ValidationResult;
8   generate(config: any, targetDir?: string): Promise<void>;
9   up(targetDir?: string): Promise<void>;
10  down(targetDir?: string): Promise<void>;
11  status(targetDir?: string): Promise<string>;
12 }
```

`validate()` gives each engine control over its own schema and error formatting. `generate()` owns the conversion from a declarative config to runnable artifacts. `up()`, `down()`, and `status()` cover the minimum lifecycle surface that both engines must support. For classic Fabric, these will delegate to the existing generated `fabric-docker.sh` scripts. For Fabric-X, they will call Docker Compose directly and check for generated files first.

The FabricXEngine will use AJV for JSON schema validation against `fabricx-schema-v1.json`, ensuring type safety and error reporting. Templates will be rendered using EJS, with context variables passed directly from the validated config.

4.4 The config schema design

The current Fabric-X schema is intentionally narrow. It aims to make the bundled local topology ergonomic first, while reserving room for later decomposition. Every field in the sample config below comes from files actually present in this workspace: the `$schema` constant from `fabricx-schema-v1.json`, the `global.tls` flag from the current engine and templates, the xdev image lineage from `compose-xdev.yml`, and the node role set from the TypeScript types and generated `core.yaml` template.

Fabric-X network config (fablo-config-fabricx.json)

```
1 {
2   "$schema": "https://fablo.io/schemas/fabricx-schema-v1.json",
3   "global": {
4     "fabricVersion": "x",
5     "tls": false
6   },
7   "fabricx": {
8     "channelId": "mychannel",
9     "namespace": "token_namespace",
10    "infrastructure": {
11      "image": "ghcr.io/hyperledger/fabric-x-committer-test-node:0.1.7",
12      "ports": {
13        "orderer": 7050,
14        "sidecar": 4001,
15        "query": 7001,
16        "database": 5433
17      }
18    },
19    "nodes": [
20      { "id": "issuer1", "type": "issuer", "apiPort": 9200, "p2pPort": 9201 },
21      { "id": "endorser1", "type": "endorser", "apiPort": 9300, "p2pPort": 9301 },
22      { "id": "owner1", "type": "owner", "apiPort": 9400, "p2pPort": 9401 }
23    ]
24  }
25 }
```

This design is intentionally honest about what it is not yet modeling. There is no attempt to flatten decomposed orderer groups, validator services, or namespace generation settings into the MVP schema prematurely. Those belong in the next phase of the mentorship, where I will extend the config model for richer topology, stronger identity handling, and broader operational settings. The result is a schema that is small enough to teach, but structurally aligned with Fabric-X from the start.

The schema enforces required fields like `channelId`, `namespace`, and `nodes`, with optional `wallets` for each node to support identity management.

4.5 Generated artifacts and docker-compose structure

The MVP does not generate a cluster of separate Fabric-X service containers yet. Instead, it generates the smallest working set of artifacts needed for the bundled `xdev` path. That includes a Docker Compose file for the infrastructure container, a routing table, one node configuration per declared logical node (including specific `core.yaml` files for Fabric-X FSC nodes and integrating orderer configuration into the Docker Compose setup), copied or future-generated crypto material, and an engine state file used by lifecycle commands. This is a sound first merge because it proves the end-to-end workflow without pretending the bundled topology is the same as the eventual decomposed one.

Generated docker-compose.xdev.yaml (excerpt)

```

1 services:
2   committer:
3     image: ghcr.io/hyperledger/fabric-x-committer-test-node:0.1.7
4     container_name: committer
5     networks:
6       - fabricx
7     environment:
8       - SC_SIDE CAR_ORDERER_IDENTITY_MSP_DIR=/root/config/crypto/peerOrganizations/
9         org1.example.com/users/Admin@org1.example.com/msp
10      - SC_SIDE CAR_ORDERER_IDENTITY_MSP_ID=Org1MSP
11      - SC_SIDE CAR_ORDERER_CHANNEL_ID=mychannel
12      - SC_SIDE CAR_ORDERER_SIGNED_ENVELOPES=true
13      - SC_QUERY_SERVICE_SERVER_ENDPOINT=:7001
14      - SC_ORDERER_BLOCK_SIZE=1
15     ports:
16       - "4001:4001"
17       - "7001:7001"
18       - "7050:7050"
19       - "5433:5433"
20     volumes:
21       - ./crypto:/root/config/crypto
22     command: run db orderer committer

```

This compose excerpt is complemented by `routing-config.yaml`, which enumerates node IDs and API endpoints, and by per-node `core.yaml` files. The generated node configuration wires FSC identity, websocket P2P addresses, sqlite persistence, endpoint resolvers for every peer node, Fabric-X driver selection, sidecar and query addresses, and token-management settings such as channel and namespace. The result is enough for `fablo generate` to materialize a coherent local developer environment from one JSON file.

[DIAGRAM PLACEHOLDER: fablo-config-fabricx.json → FabricXEngine.generate() → docker-compose.xdev.yaml + routing-config.yaml + per-node core.yaml → fablo up → running Fabric-X network with port-readiness polling]

Figure 2: Fablo-FabricX integration: config to running network

5 MVP Plan and Timeline

5.1 Phase plan and milestone table

This schedule reflects the need to implement the pluggable engine architecture from scratch while building on Fablo's existing foundation. Phase 1 focuses on creating the engine framework and the initial Fabric-X engine implementation. The midterm evaluation provides a concrete demonstration of the architecture

working end-to-end. Subsequent phases extend the crypto generation, topology support, and hardening.

5.2 Gantt

placeholder

[DIAGRAM PLACEHOLDER: Gantt chart — June through November timeline showing phases, milestones, and dependencies]

Figure 3: Project Gantt chart

The dependency structure behind the timeline is simple and defensible. Schema design depends on settled engine boundaries. Reliable lifecycle commands depend on stable generated artifacts. Docker-orchestrated crypto generation depends on understanding the existing Fabric-X sample toolchain. Decomposed topology support depends on first proving the engine model against `xdev`. Documentation and community adoption depend on all of the above.

6 Full Technical Approach: Beyond the MVP

6.1 Crypto material generation for Fabric-X

The MVP copies crypto material from sample locations because that is the smallest path to proving the engine contract. The full implementation should replace that with a Fabric-X-aware crypto pipeline built into the Fabric-X engine. The local sample script

`fabric-x-repo/samples/tokens/scripts/gen_crypto.sh` provides the roadmap. It starts a Fabric CA server, enrolls a CA admin, registers and enrolls owner identities using Idemix, enrolls TLS identities for FSC nodes, and finally runs `go tool tokengen gen zkstdlognogh.v1` to generate issuer and namespace parameters. That script is not optional sample decoration; it is the clearest current specification of how Fabric-X local identities are assembled.

The critical UX constraint is that this pipeline cannot depend on host-installed Go or Fabric-X binaries.

Fablo's promise is that the user needs Docker and Node.js, not a handcrafted local toolchain. The right implementation is therefore Docker-orchestrated crypto generation. `FabricXEngine.generate()` should spin up ephemeral utility containers that mount the target directory, run the Fabric-X crypto scripts and binaries inside the container image, write artifacts into mounted volumes, and then exit. The user sees one `fablo generate` command; inside that command the engine runs one-off containers for `cryptogen`, Fabric CA enrollment, and `tokengen` steps.

I break this into three stages, and that is still the right structure. Stage one generates classic Fabric MSP and TLS material for orderer-side compatibility inside a utility container. Stage two generates FSC node identities and owner identities using Fabric CA enrollment logic derived from `gen_crypto.sh`, again inside a disposable container or container pair. Stage three generates token namespace parameters with `tokengen` in a dedicated tool container. Each stage should have explicit output directories and validation checks so the engine can fail fast when a required artifact is missing.

Planned crypto generation contract

```
1 export interface FabricXCryptoOutputs {
2   ordererMspDir: string;
3   ordererMspId: string;
4   nodeCertDirs: Record<string, string>;
5   ownerWalletDirs: Record<string, string>;
6   namespaceDir: string;
7   genesisBlockPath?: string;
8 }
9
10 export interface FabricXCryptoGenerator {
11   generate(config: FabricXConfig, targetDir: string): Promise<
12     FabricXCryptoOutputs>;
13   validate(outputs: FabricXCryptoOutputs): Promise<void>;
14 }
```

This keeps the complexity isolated. The command layer still sees only `generate()`, but inside the Fabric-X engine the generator evolves from “copy known crypto” to “run disposable tool containers and validate required identities”. That path is also consistent with the local Fabric-X version story. The current sample image is pinned to 0.1.7, while `fabric-x-repo/go.mod` already depends on a newer committer lineage. Making crypto generation an explicit Docker-orchestrated module rather than a pile of copied files will make that version drift easier to manage.

6.2 Multi-service

topology

support

The current schema is effectively `xdev`-only. The Ansible inventory makes clear that a realistic Fabric-X topology includes separate orderer groups and separate committer family services. The next schema revision should therefore add an explicit topology mode and a structured infrastructure section for decomposed services. The important design point is that this should extend the existing Fabric-X section rather than reintroduce classic Fabric abstractions under different names.

The routing layer is where this becomes operationally real. Today, `routing-config.yaml.ejs` iterates over `nodes` and emits one route per FSC node. In the decomposed model, the generator has to emit both FSC-node routes and service-family endpoints so that sidecars and application-facing nodes can resolve the right infrastructure addresses for endorsers, committers, and query services. In practice, this means the EJS templates will map user-declared nodes such as `endorser1` or `owner1` together with decomposed service definitions into a deterministic routing table. The config is the source of truth; the generated routing file is a derived artifact.

Planned multi-service Fabric-X config (illustrative)

```
1 {
2   "$schema": "https://fablo.io/schemas/fabricx-schema-v2.json",
3   "global": { "fabricVersion": "x", "tls": true },
4   "fabricx": {
5     "topology": "decomposed",
6     "channelId": "arma",
7     "namespace": "token_namespace",
8     "ordererOrgs": [
9       {
10        "name": "OrdererOrg1",
11        "domain": "ordererorg1.example.com",
12        "group": 1,
13        "routerPort": 7050,
14        "batcherPort": 7051,
15        "consenterPort": 7052,
16        "assemblerPort": 7053
17      }
18    ],
19    "committer": {
20      "databasePort": 5435,
21      "validatorPort": 5100,
22      "verifierPort": 5200,
23      "coordinatorPort": 5300,
24      "sidecarPort": 4001,
25      "queryPort": 7001
26    },
27    "nodes": [
28      { "id": "issuer1", "type": "issuer", "apiPort": 9200, "p2pPort": 9201 },
29      { "id": "endorser1", "type": "endorser", "apiPort": 9300, "p2pPort": 9301 },
30      { "id": "owner1", "type": "owner", "apiPort": 9400, "p2pPort": 9401 }
31    ]
32  }
33 }
```

I should not try to ship all of this at once. The correct sequencing is to land the **xdev** engine first, then add decomposed Docker Compose support for one topology slice, most likely the committer family, before tackling multi-organization orderer groups. This is both technically safer and easier to review upstream.

6.3 CCaaS and endorser integration

Classic Fablo already treats chaincode lifecycle as a first-class concern, and Fabric-X needs an equivalent story rather than a silent omission. The relevant difference is that Fabric-X's execution path is centered on the endorser microservice and Chaincode-as-a-Service style connectivity instead of the classic peer lifecycle. That means the Fabric-X config model eventually needs a dedicated section for contract execution inputs, not just infrastructure and node topology.

The practical MVP extension is to let the Fabric-X config declare one or more CCaaS contracts with a local source directory, image or build instructions, and the endpoint that the endorser-side execution environment should use. Fablo should then generate the necessary Docker Compose services or mounts so a developer can bind a local chaincode directory into the Fabric-X execution environment in the same spirit as the existing classic dev-mode and CCaaS support. The goal is to preserve the Fablo user expectation that a local project directory can be part of the declarative network description.

Planned CCaaS extension for Fabric-X

```
1 {
2   "fabricx": {
3     "chaincodes": [
4       {
5         "name": "tokencc",
6         "lang": "ccaas",
7         "directory": "./chaincodes/tokencc",
8         "endpoint": "tokencc:9999",
9         "endorser": "endorser1"
10      }
11    ]
12  }
13 }
```

This is not only a convenience feature. It is required if the Fabric-X path is supposed to support realistic developer testing. A local network that can boot but cannot attach and exercise a contract is incomplete from the perspective of Fablo users.

6.4 Full e2e test suite

The Fabric-X test strategy should consciously mimic the classic Fablo style rather than inventing a new harness. The repository already shows the pattern in `e2e-network/docker/test-01-v3-simple.sh`: create a temp workspace, build, initialize, start, wait, assert, mutate, and tear down. The existing Fabric-X test in `e2e-network/fabricx/test-01-fabricx-simple.sh` is already aligned with that spirit, and the mentorship should grow it into a small matrix of focused scenarios: generation-only validation, happy-path xdev lifecycle, missing-crypto behavior, config validation failures, and eventually decomposed topology bootstrap.

E2E test: test-01-fabricx-simple.sh (full structure)

```
1  #!/usr/bin/env bash
2  # test-01-fabricx-simple.sh
3  # Validates: single-org xdev Fabric-X network bootstraps and responds
4
5  set -euo pipefail
6
7  ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../../.." && pwd)"
8  WORKDIR="$ROOT_DIR/e2e-network/_tmp_/fabricx-test-01"
9
10 rm -rf "$WORKDIR"
11 mkdir -p "$WORKDIR"
12 cd "$WORKDIR"
13
14 echo "==_Build_"
15 (cd "$ROOT_DIR" && npm run build >/dev/null)
16
17 echo "==_Init_"
18 npx fablo init --fabricx
19 test -f fablo-config-fabricx.json
20 grep -q 'fabricx-schema-v1.json' fablo-config-fabricx.json
21
22 CONFIG_FILE="$WORKDIR/fablo-config-fabricx.json"
23 TARGET_DIR="$WORKDIR/fablo-target/fabricx"
24
25 echo "==_Generate_"
26 npx fablo generate "$CONFIG_FILE" "$TARGET_DIR"
27 test -f "$TARGET_DIR/docker-compose.xdev.yaml"
28 test -f "$TARGET_DIR/conf/routing-config.yaml"
29 test -f "$TARGET_DIR/conf/issuer1/core.yaml"
30
31 echo "==_Up_"
32 FABLO_LOCAL=true npx fablo up "$CONFIG_FILE" "$TARGET_DIR"
33
34 echo "==_Wait_For_Ports_"
35 for port in 7050 7001; do
36   for i in $(seq 1 60); do
37     nc -z 127.0.0.1 "$port" && break
38     sleep 1
39   done
40   nc -z 127.0.0.1 "$port" || exit 1
41 done
42
43 echo "==_Status_"
44 npx fablo status "$CONFIG_FILE" "$TARGET_DIR" | grep -q 'committer'
45
46 echo "==_Down_"
47 npx fablo down "$CONFIG_FILE" "$TARGET_DIR"
48 docker ps --filter name=committer --format '{{.Names}}' | grep -q '^committer$' &&
49   exit 1
50 echo "ALL_TESTS_PASSED"
```

This structure is intentionally simple. It proves the exact user contract the mentorship cares about. More ambitious functional token-flow checks can be added later, but the first responsibility of the test suite is to guarantee that Fablo can reliably bootstrap and tear down a Fabric-X network without regressing classic Fabric behavior.

6.5 Lifecycle management: reset and channel tooling

The current interface does not yet include a Fabric-X-specific `reset()` method, but the project scope should. Classic Fabric reset is largely about containers, chaincode state, and generated network artifacts. Fabric-X reset has to reason about a different state surface: Docker volumes, the bundled database state, generated per-node configuration, generated crypto material, generated namespace artifacts, and possible CCaaS-related containers or mounts. A bare `docker compose down -v` is not strict enough on its own.

Planned Fabric-X reset flow

```
1 async reset(targetDir?: string): Promise<void> {
2   const dir = resolveTargetDir(targetDir);
3   const composeFile = path.join(dir, "docker-compose.xdev.yaml");
4
5   if (await fs.pathExists(composeFile)) {
6     execSync(`docker compose -f ${composeFile} down -v`, { stdio: "inherit"
7       });
8   }
9
10  execSync(`docker container prune -f`, { stdio: "inherit" });
11  await fs.remove(path.join(dir, "conf"));
12  await fs.remove(path.join(dir, "crypto"));
13  await fs.remove(path.join(dir, "data"));
14  await fs.remove(path.join(dir, "namespace"));
15  await fs.remove(path.join(dir, "wallets"));
16  await fs.remove(path.join(dir, "fabricx-engine-state.json"));
17 }
```

The cleanup contract should be explicit in the documentation: `fablo reset` for Fabric-X tears the stack down, removes generated `conf/`, `crypto/`, `data/`, `namespace/`, and wallet state under the target directory, and cleans up container state associated with the generated network. Channel tooling is a related future concern. Classic Fablo already exposes channel query helpers because classic Fabric has a channel administration surface worth scripting. Fabric-X has its own namespace and configuration lifecycle through `fxconfig`. The right medium-term direction is not to shoehorn classic channel tooling into Fabric-X, but to add Fabric-X-aware helper commands once the bootstrap engine is stable.

7 Known Limitations and How They Will Be Resolved

Known MVP Limitations (all isolated inside the new engine architecture)

- **Engine architecture is implemented from scratch.** The current Fablo codebase uses monolithic Docker-based generation. The MVP introduces the pluggable engine system, which requires careful integration to maintain backward compatibility with existing classic Fabric workflows.
- **Crypto material is copied, not generated.** The Fabric-X engine will initially copy crypto from sample locations. The post-MVP plan replaces this with a staged crypto-generation pipeline using Docker containers for `cryptogen`, Fabric CA enrollment, and `tokengen`.
- **Only xdev bundled topology is supported.** The initial Fabric-X engine generates artifacts for the bundled `xdev` topology (single container with db, orderer, committer). The post-MVP plan extends the schema and engine to support decomposed multi-service topologies.
- **Orderer identity defaults are sample-based.** The initial implementation uses environment variables derived from sample crypto. The post-MVP plan promotes orderer identity management into explicit configuration and generated crypto outputs.
- **Limited lifecycle tooling initially.** The MVP focuses on core `generate`, `up`, `down`, and `status` commands. Advanced features like Fabric-X-specific reset and namespace tooling are deferred to post-MVP phases.

None of these limitations affect existing classic Fabric functionality.

The important point is not that the MVP has limitations; all credible MVPs do. The important point is that each limitation is already isolated and explained. The engine split ensures that classic Fabric behavior remains unaffected. I have also defined the replacement path for each shortcut directly in this proposal, which keeps the mentorship plan reviewable and lowers the risk of architectural drift.

8 Pre-Application

Research

Pre-Application Research Completed

Before writing this proposal, the following research was completed on the current Fablo codebase:

- Analyzed the current Docker-based generation architecture in `src/setup-docker/index.ts`, `src/commands/setup-network/index.ts`, and the EJS template system in `src/setup-docker/templates/`; identified the need for pluggable engines to support different network types.
- Studied the Oclif command structure and lifecycle management through generated shell scripts; confirmed that lifecycle operations can be abstracted into engine methods.
- Researched Fabric-X architecture through `fabric-x-repo/README.md`, `samples/tokens/compose-xdev.yml`, `samples/tokens/ansible/inventory/fabric-x.yml`, and crypto generation scripts; understood the gap between bundled local topology and decomposed production topology.
- Designed the pluggable engine interface and schema-first detection mechanism to enable clean extension of Fablo for Fabric-X while maintaining backward compatibility.
- Identified key architectural decisions: engine interface design, schema versioning strategy, template organization, lifecycle abstraction, and backward compatibility approach.
- Validated the approach through analysis of existing patterns in Fablo's codebase and alignment with Fabric-X requirements.

This pre-application work is important because it changes the proposal from speculative to evidence-based. The working PoC is not presented as final software, but it is strong proof that the architectural direction is sound and that the candidate has already navigated the difficult parts of the codebase.

9 Why I Am the Right Candidate

My fit for this project comes from the overlap between implementation skills and evidence of already having done the architectural homework. The Fabric-X plus Fablo integration is not a narrow blockchain task. It requires TypeScript CLI work, schema design, template generation, Docker runtime understanding, test automation, and careful reading of a second codebase whose core tooling is in Go. That blend matches my current profile well. My background in TypeScript, JavaScript, backend systems, and Docker-based workflows means I am comfortable making changes across the exact layers this project touches.

The strongest evidence is the work already present in my proof of concept and in the implementation described throughout this proposal. I did not stop at reading READMEs and writing a high-level plan. I implemented the engine split cleanup tasks, updated command routing, hardened the Fabric-X generator, fixed preconditions, aligned TLS handling, tightened schema and type boundaries, improved test coverage, and worked through the open design questions that still matter. The architectural diagrams I created were not decorative; they were part of understanding how the current implementation behaves and where its seams are.

My project history also maps cleanly onto the needs of this mentorship. Campus Marketplace involved full-stack system design, authentication, RBAC, and hybrid data-layer work, which is relevant because

Fablo integration work is fundamentally about turning a user-facing developer surface into correct infrastructure behavior. AuthrNet is relevant because it required blockchain-oriented reasoning, identity flows, and coordinating a user-facing application with decentralized primitives. My internship and hackathon backend work are relevant because they demonstrate a pattern of working across interfaces, services, and deployment environments instead of remaining inside a single feature layer.

Open source fit matters just as much as raw technical fit. This project needs someone who can separate what should be merged now from what should be deferred, document known limitations honestly, and structure work so maintainers can review it in small, coherent pieces. The existing proof of concept, research notes, and documentation updates show that I approach the work that way. I am already treating the Fabric-X path as something that must preserve classic Fabric behavior, maintain a clean extension seam, and stay reviewable by upstream maintainers. That mindset is as important as writing code quickly.

10 Risk Register

The most important feature of this risk register is that I tie every item to something directly observed in the repositories rather than to generic software-project uncertainty. Version drift is visible already between the sample image lineage and current Go module state. Crypto complexity is visible in the sample shell script. Topology mismatch is visible between `compose-xdev.yml` and the Ansible inventory. Those are real risks, which makes them manageable.

11 Success Metrics

Core Deliverables — all must be complete

- ✓ Design proposal document accepted and merged
- ✓ `fablo init -fabricx` generates valid, schema-identified config
- ✓ `fablo generate` produces complete docker-compose, routing-config, and per-node core.yaml
- ✓ `fablo up / down / status` lifecycle working end-to-end
- ✓ E2E test suite passing, following existing Fablo test structure
- ✓ Contributor and user documentation merged
- ✓ At least two community members have successfully run a Fabric-X network using the implementation

Stretch Goals — conditional on core completion by Week 12

- Multi-org Fabric-X topology support
- Separate-container committer, validator, verifier, coordinator, and orderer deployment
- CI integration example for Fabric-X networks
- Fablo UI support for Fabric-X configs and topology visualization

colorbox

Dates	Phase	Deliverables
June 01–June 14	Onboarding	Finalize the design proposal upstream, set up the local development environment, confirm PR review expectations with mentors, and rehearse the existing build and e2e harness so new Fabric-X work lands in the same workflow as classic Fablo changes.
June 15–July 31	Phase 1: Implement Pluggable Engine Architecture	Create the FabloEngine interface and engine loader system; implement schema-first engine detection; develop the ClassicFabricEngine wrapper for existing Docker-based generation; create the FabricXEngine with AJV validation against fabricx-schema-v1.json ; implement EJS templates for Fabric-X Docker Compose and configuration files; add fablo init -fabricx command; ensure backward compatibility with existing classic Fabric workflows.
August 01–August 23	Phase 2: Lifecycle and Local Bootstrap	Wire up fablo up , fablo down , and fablo status against generated Fabric-X artifacts; finalize standard port polling, TLS handling, and precondition checks; and harden e2e-network/fabricx/test-01-fabricx-simple.sh so it validates the Docker Compose stack starts and tears down correctly, mirroring the existing test-01-simple.sh style E2E testing patterns.
August 24–August 31	Midterm Evaluation	Deliver a working, mergeable PR that demonstrates the pluggable engine architecture with Fabric-X support for the bundled xdev topology, including schema validation, generated Docker assets, lifecycle commands, and automated e2e tests.
September 01–October 15	Phase 3: The Crypto Pipeline and Routing	Replace copied sample crypto with dynamic generation orchestrated from Docker utility containers, generate token namespace parameters with tokenngen , formalize orderer identity inputs and outputs, and deepen the generated routing layer so node and sidecar connectivity are driven directly from the Fabric-X config model.
October 16–November 14	Phase 4: Multi-Service Topology and Hardening	Introduce topology: decomposed support for separate orderer, endorser, committer-family, and query-service containers; add stricter reset semantics and CCaaS-aware cleanup; and finish contributor documentation, user guides, and architectural decision records.
November 15–November 30	Final Evaluation	Reserve buffer for final upstream review and integration work. Only if the prior phases are fully merged, use this period for stretch goals such as CI integration examples or Fablo UI exposure for Fabric-X networks.

Table 3: Official LFX timeline mapped to project milestones

Area	Relevant experience
TypeScript / Node.js	Built full-stack systems in JavaScript and TypeScript, including Campus Marketplace and AuthrNet; already modified Fablo's TypeScript command, engine, and template-generation code.
Go	Read and reasoned through Fabric-X's Go-based <code>fxconfig</code> tooling, <code>go.mod</code> , and integration tests; experienced enough with Go ecosystems to understand versioning, module boundaries, and CLI tool integration.
Bash / CLI tooling	Comfortable with shell-driven workflows through e2e test reading and implementation work; the Fabric-X proof of concept already extends Fablo CLI behavior and its Bash-based test path.
Docker / compose	Extensive Docker and GitHub Actions use in prior projects; directly relevant to compose generation, multi-service orchestration, and startup-debugging for Fabric-X networks.
Distributed systems	Worked on decentralized and backend-heavy projects; understands multi-service startup ordering, identity boundaries, and infrastructure debugging, which are central to Fabric-X.
Open source	More than 800 GitHub contributions, FOSS mentoring, and active collaborative development experience; relevant to delivering reviewable, maintainable work in an upstream community.
Debugging multi-service systems	Experience integrating APIs, services, authentication, databases, and deployment tooling; the current Fabric-X work already involved tracing runtime assumptions across generated config, Docker Compose, and sample crypto layout.

Table 4: Skills and experience mapping to project requirements

Risk	Mitigation strategy	Probability
Fabric-X API surfaces change before stable release	Keep version-sensitive behavior isolated inside <code>src/engines/fabricx/</code> ; pin sample images explicitly; document a compatibility profile strategy so schema contracts can remain stable across runtime image updates.	Medium
Crypto generation for FSC nodes is under-documented	Treat <code>samples/tokens/scripts/gen_crypto.sh</code> as the executable reference, implement the crypto pipeline in stages, and keep a fallback path for externally supplied crypto material during transition.	High
<code>xdev</code> bundled topology differs from final decomposed topology	Declare <code>xdev</code> as the MVP scope, design the schema with explicit topology evolution in mind, and add decomposed support only after the bundled engine path is merged and tested.	Medium
Docker networking for decomposed services is complex	Start from the known Ansible inventory and bundled sample port map; introduce decomposed services incrementally rather than as a single large compose rewrite.	Low
Stretch goals crowd core work	Core must be merged before Week 13 stretch begins; use the engine boundary and documented limitation list to keep scope decisions explicit.	Medium (mitigated)

Table 5: Risk register with mitigation strategies